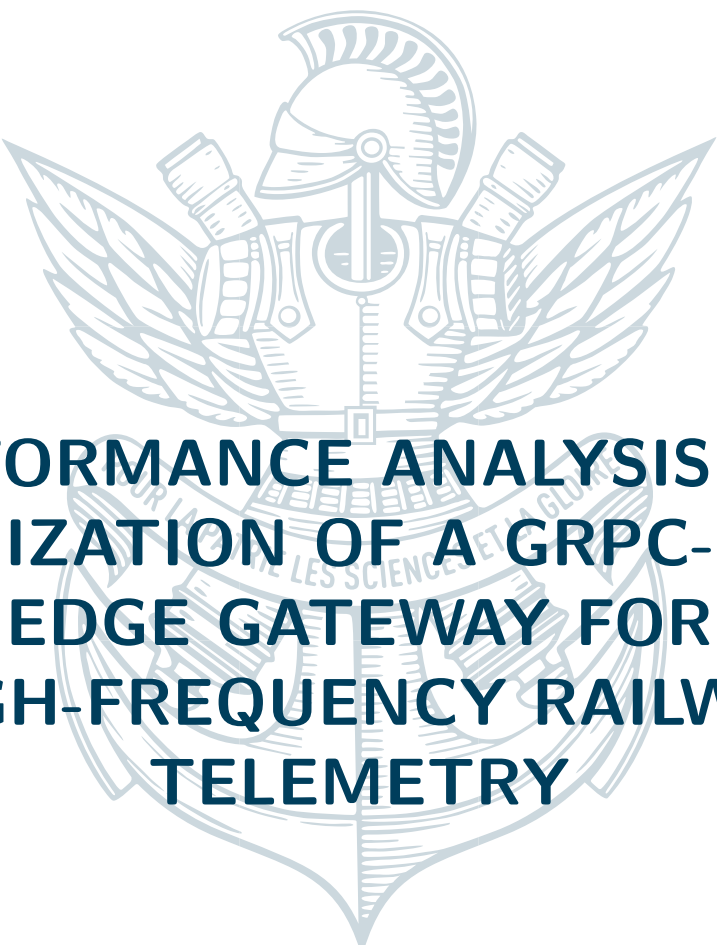




PERFORMANCE ANALYSIS AND OPTIMIZATION OF A GRPC-BASED EDGE GATEWAY FOR HIGH-FREQUENCY RAILWAY TELEMETRY

March 26, 2026



Zehua HUANG



1

ABSTRACT

Abstract

This study assesses the performance and scalability of a Go-implemented gRPC edge gateway tailored for high-frequency telemetry within a 32-kilometer railway monitoring network. The system is charged with processing real-time vibration and temperature data from 16 edge devices, targeting an aggregate throughput in the 8 Gbps range.

Preliminary benchmarking indicates that while the baseline setup effectively manages low-load conditions, it undergoes substantial performance degradation under heavy traffic. High-load stress testing results show that the proposed Optimized-Unary mode delivers a 2.14-fold throughput improvement, rising from 477.1 to 1,122.3 requests/s. Additionally, this optimization effectively mitigates latency spikes, cutting average and P95 latency by 50% and 56% respectively (from 25.72 ms and 33.65 ms to 12.75 ms and 14.79 ms), while maximum tail latency decreases by 50.5% (from 97.44 ms to 48.26 ms).

Technical analysis pinpoint the primary bottlenecks as elevated CPU overhead stemming from intensive string-based map serialization (`map<string, double>`) and frequent Garbage Collection (GC) cycles induced by short-lived object allocation. To address these challenges, this research introduces a tiered optimization approach encompassing traffic differentiation, data model enhancement through compact repeated structures, and the deployment of `sync.Pool` to minimize memory usage. These optimizations stabilize tail latency and enhance system reliability, establishing a solid foundation for large-scale, safety-critical railway IoT implementations.

Keywords: gRPC, Edge Computing, Railway IoT, Performance Optimization, High-Frequency Telemetry, Tail Latency, Go.

2

INTRODUCTION

2.1 BACKGROUND AND MOTIVATION

The global expansion of High-Speed Railway (HSR) networks has transformed infrastructure maintenance practices, shifting from reactive manual inspections to proactive, real-time structural health monitoring. To guarantee operational safety at extreme velocities, contemporary systems employ Distributed Acoustic Sensing (DAS), which converts existing fiber-optic cables installed alongside railway tracks into thousands of virtual vibration sensors.

A key technical benefit of this approach is the dual functionality of optical fiber infrastructure:

1. Sensing capability: The fiber operates as a continuous sensor detecting sub-nanometer strains and acoustic patterns, delivering comprehensive insights into rail conditions and structural integrity.
2. Communication capacity: The same fiber functions as a high-speed backhaul backbone, employing Wave Division Multiplexing (WDM) to transmit extensive telemetry data through gRPC.

Despite these advantages, the system encounters significant data processing challenges. Each telemetry message can include over 2,000 vibration samples, and with 16 or more devices uploading simultaneously, the gateway must manage an aggregate throughput in the 8 Gbps range. Moreover, during “train-passing” events, data becomes substantially more time-critical, necessitating ultra-low latency for real-time anomaly detection and GPU-based inference. This research is driven by the need to reconcile massive data volumes with stringent real-time requirements through an optimized gRPC gateway architecture.

2.2 PROBLEM STATEMENT: DATA AND ARCHITECTURAL BOTTLENECKS

While DAS delivers exceptional sensing capabilities, the conversion from raw optical signals to actionable intelligence encounters three critical bottlenecks:

- Elevated Serialization Overhead: A single telemetry message can include over 2,000 vibration sampling points. In the baseline implementation, employing `map<string, double>` structures introduces substantial CPU overhead since each vibration point carries a string-based key. This results in a serialization volume of approximately 40 KB per message, which is inefficient for high-frequency updates.

- **Concurrency and Resource Constraints:** In a typical railway monitoring scenario, 16 or more edge devices may upload data concurrently, targeting aggregate throughput in the 8 Gbps range. This multi-device concurrency triggers frequent Garbage Collection (GC) cycles due to short-lived object allocations, causing unstable tail latency and potential server-side blocking.
- **Static Transmission vs. Dynamic Priority:** Conventional gateways often utilize a static "full-fidelity" transmission mode. However, railway telemetry is context-dependent: real-time priority must increase substantially during "train-passing" periods. A simplistic synchronous pipeline fails to distinguish between idle background noise and critical vibration events, squandering bandwidth and compute resources during normal operation.

2.3 THE ROLE OF gRPC IN AI-DRIVEN RAILWAY ANALYTICS

Contemporary DAS systems necessitate the integration of Artificial Intelligence (AI) for real-time signal classification, such as detecting rail cracks or identifying locomotive types. These inference tasks are computationally demanding, typically requiring a 10 ms delay per request to simulate GPU processing.

To bridge the gap between high-speed data acquisition and resource-intensive analytics, we employ gRPC as the communication backbone. gRPC's language-independent interface enables the gateway to efficiently "push" telemetry to diverse backend services. However, to achieve the 8 Gbps target, the standard unary RPC model requires enhancement. This research concentrates on optimizing the gRPC layer through compact Protobuf encoding, object pooling via `sync.Pool`, and dual-priority worker pools to ensure that real-time AI inference does not become a system bottleneck.

2.4 PROPOSED SOLUTION: OPTIMIZED EDGE GATEWAY ARCHITECTURE

To tackle the identified bottlenecks, this project implements a specialized Edge Gateway designed to separate high-frequency data acquisition from intensive downstream GPU processing. The gateway functions as a high-performance intermediary, employing a tiered optimization approach to efficiently manage telemetry ingestion and resource allocation.

A key feature of the gateway is a server-driven adaptive state machine that dynamically switches operational modes based on real-time "vibration loudness":

- **ACTIVE Mode:** Activated during train-passing events, the system prioritizes real-time responsiveness. In this state, the gateway enables low-latency unary or streaming paths to ensure critical signals are processed with minimal delay.

- **HEARTBEAT Mode:** During idle periods (vibration amplitude ≈ 0.1), the gateway transitions to a resource-efficient state. Telemetry is aggregated through temporal batching (e.g., batch size of 5) to reduce RPC frequency, thereby minimizing server-side CPU interrupts and connection overhead.

2.5 COMMUNICATION PROTOCOL AND RPC DESIGN

To achieve high-throughput stability, we evolve from a basic baseline to an optimized gRPC framework. The redesign focuses on three key dimensions: data compression, memory management, and transmission patterns.

- **Data Model Enhancement (Primary Driver):** The baseline implementation uses a `map<string, double>` structure, which generates significant serialization overhead due to repeated string keys and 8-byte values. Our optimized version replaces this with a compact repeated message (`uint32 id` and `float value`), reducing payload size by approximately 55% (from 40 KB to 18 KB per 2,000 samples).
- **Memory Management through `sync.Pool`:** To alleviate Garbage Collection (GC) pressure under 8 Gbps traffic, the gateway implements object pooling. By reusing pre-allocated structures for request handling, the system substantially reduces allocation-related latency fluctuations.
- **Transmission Variants:** We assess four optimized patterns against the baseline:
 1. **Optimized Unary:** Maintains simplicity while leveraging compact encoding to achieve a 2.14-fold throughput improvement.
 2. **Batch Mode:** Decreases per-call metadata overhead by buffering multiple samples, ideal for **HEARTBEAT** phases.
 3. **Streaming gRPC:** Enables continuous data flow for equitable protocol comparison and sequencing.
 4. **Mixed Mode:** A server-streaming-driven adaptive model that uses `SubscribeModeChanges` to direct clients to switch between Batch and Unary modes in real-time.

2.6 SCOPE OF WORK

This research systematically evaluates the design and performance of the optimized gRPC gateway. The primary areas of focus include:

1. **Data Protocol Optimization:** The design of `.proto` structures evolving from `map<string, double>` to compact `repeated Metric` formats to minimize serialization overhead.

2. High-Concurrency Server Architecture: Implementation of internal mechanisms including dual-priority queuing, a 64-worker execution pool, and memory reuse via `sync.Pool` to stabilize performance under heavy load.
3. Adaptive Transmission Logic: The development of a server-driven state machine that employs vibration loudness thresholds to dynamically switch clients between **ACTIVE** (low-latency) and **HEARTBEAT** (resource-efficient) modes.
4. Performance Evaluation: A comprehensive comparative analysis of five transmission variants (Baseline, Optimized Unary, Batch, Stream, and Mixed) under simulated 8 Gbps aggregate loads with 16 concurrent devices.

3 RELATED WORK

The design of a high-performance edge gateway for High-Speed Railway (HSR) monitoring exists at the intersection of fiber-optic sensing, edge computing, and high-throughput communication protocols. This section reviews existing research and identifies the architectural gaps addressed by this project.

3.1 DAS AND RAILWAY STRUCTURAL HEALTH MONITORING (SHM)

Distributed Acoustic Sensing (DAS) has emerged as a transformative technology for HSR, leveraging existing fiber-optic infrastructure as a continuous array of virtual vibration sensors. By analyzing Rayleigh backscattered light, DAS achieves sub-nanometer strain sensitivity over extensive distances [1]. However, many existing deployments utilize a tightly coupled, monolithic architecture where DAS interrogators are co-located with local GPU processing units. This "siloe" approach complicates maintenance and restricts independent scalability of sensing and computing layers. In contrast, this project implements a gateway functioning as an independent service node, decoupling the DAS client from the GPU backend to enable modular hardware upgrades and enhance system resilience.

3.2 EDGE COMPUTING AND INDUSTRIAL IIOT GATEWAYS

Edge computing is crucial for HSR monitoring as it provides low-latency pre-processing near the trackside. Traditional Industrial IIOT (IIoT) gateways primarily focus on protocol

translation (e.g., MQTT to OPC-UA). While effective for low-bandwidth sensors, they are not optimized for the 8 Gbps aggregate throughput inherent to DAS. Most existing gateways offer generic "always-on" forwarding without context-aware flow control. Our work extends the traditional gateway by introducing priority queuing and a hybrid stream/batch API to differentiate between real-time train traffic and background monitoring, moving beyond simple forwarding to intelligent stream orchestration.

3.3 COMMUNICATION PROTOCOLS AND STREAMING IN IoT

The limitations of traditional IoT protocols, such as the per-packet header overhead of MQTT or HTTP/1.1, are well-documented in high-frequency telemetry scenarios. gRPC, built on HTTP/2, provides a high-performance alternative through binary serialization. Recent empirical evaluations, such as the study by [?], have demonstrated that gRPC significantly outperforms REST-based architectures in communication efficiency, particularly within microservice-oriented ecosystems where high-frequency data exchange is prevalent.

While generic gRPC studies often concentrate on simple unary calls, our work implements an optimized streaming and batching model. This architecture utilizes client-streaming for high-throughput telemetry and a separate server-streaming RPC for control (mode-switching). This design ensures that command latency is decoupled from heavy data telemetry, enabling the system to handle 8 Gbps ingestion while maintaining a responsive control back-channel.

3.4 ADAPTIVE SAMPLING AND SERVER-DRIVEN MODE SWITCHING

Adaptive sampling techniques typically reduce data volume by adjusting resolution based on local event detection. Most literature focuses on client-side decision-making, which increases the computational burden on the sensor node. In our architecture, the decision logic is relocated to the gateway server, which calculates "loudness" (vibration amplitude) from incoming telemetry to drive a state machine. The server then pushes **ACTIVE** or **HEARTBEAT** instructions via the control stream. This server-driven approach ensures a "global view" of track activity while allowing the DAS device to remain a lightweight data producer.

3.5 PRIORITY SCHEDULING AND REAL-TIME PROCESSING

Priority queuing is essential to prevent mission-critical alerts from being delayed behind routine telemetry—a phenomenon often causing increased latency for urgent messages during high-load periods. The optimized gateway presented here implements a dual-channel priority queue with dedicated worker pools (e.g., prioritized workers for train events vs. background monitoring). This reduces the risk of critical traffic waiting behind bulk telemetry, ensuring

low-latency processing even under the simulated 10 ms GPU processing delay used in our benchmarks.

3.6 SUMMARY OF CONTRIBUTIONS

While existing research provides foundations for DAS sensing, edge computing, and gRPC, there is limited work integrating hardware decoupling, hybrid streaming/batching, and server-driven mode switching into a single, high-throughput HSR gateway. This project addresses this gap by providing a comprehensive, benchmarked architecture specifically tailored for the 8 Gbps demands of railway structural health monitoring.

4 SYSTEM ARCHITECTURE: BASELINE GATEWAY

4.1 PROTOCOL AND SERVICE DEFINITION: GATEWAY.PROTO

The baseline protocol is defined in `gateway.proto` and serves as a minimal implementation for telemetry ingestion. The schema centers around a single-request/single-response model with the following core components:

- **Data Structure:** Telemetry data is encapsulated in a `Telemetry` message containing:
 - `device_id` and `timestamp_ms` for identification.
 - `metrics`: A `map<string, double>` representing sensor readings. While flexible, this map-based structure generates substantial serialization and memory overhead at high frequencies.
 - `client_avg`: A pre-calculated mean for server-side verification.
- **Service Interface:** The `EdgeGatewayService` defines a standard Unary RPC, `PublishTelemetry`, alongside a basic `HealthCheck`.
- **Response Logic:** The server responds with a `TelemetryAck` containing the `status`, `server_avg`, and a `verified` boolean.

This protocol lacks support for batching, streaming, or priority tagging. Each packet requires a complete gRPC round-trip, resulting in a "chatty" protocol that is sensitive to network and processing latency.

4.2 ARCHITECTURAL PARADIGM: SYNCHRONOUS INGESTION

The Baseline Gateway is designed as a control group to evaluate the performance of standard, synchronous RPC patterns in high-throughput environments. It implements a **one-to-one mapping** between client telemetry requests and server-side **handler goroutines**. While architecturally simple, this approach introduces significant overhead due to the inherent coupling of network transmission and processing latency.

To ensure experimental rigor, the baseline service operates as a **separate process** with an **independent network listener**. This isolation enables concurrent stress testing without resource or port contention between the baseline and optimized architectural models.

4.3 EXECUTION MODEL AND GPU SIMULATION

A critical aspect of the baseline architecture is the modeling of downstream processing costs. In a production environment, DAS telemetry requires intensive AI-driven analysis for structural health monitoring. To simulate this, the baseline gateway incorporates an **inline processing bottleneck**:

- **Latency Injection:** Each incoming telemetry packet undergoes a fixed **10 ms** blocking delay within its handler goroutine. This simulates the execution time of a typical GPU-accelerated inference pipeline (e.g., vibration signature classification).
- **Blocking I/O Semantic:** In the baseline model, the gRPC handler remains occupied throughout the entire simulated processing duration. Without asynchronous buffering or worker-pool abstraction, the server's maximum theoretical throughput is strictly limited by the number of concurrent RPC handlers and the injected 10 ms latency.

4.4 PERFORMANCE BOTTLENECK ANALYSIS

Through empirical benchmarking under 8 Gbps aggregate load, the baseline gateway reveals two primary limitations of standard Unary RPCs for high-frequency DAS data ingestion:

1. **Elevated Serialization Cost and Payload Redundancy:** The most significant bottleneck is the use of `map<string, double>` for vibration data. Each of the 2,000 sampling points per message carries a string-based key, resulting in a payload of approximately 40 KB. This creates intensive CPU overhead during Protobuf serialization/deserialization [2]. Our analysis indicates that this redundant metadata accounts for nearly **55%** of the total traffic volume.

2. **Memory Pressure and GC Instability:** Under the concurrent load from 16 devices, the rapid allocation and reallocation of short-lived telemetry objects trigger frequent **Garbage Collection (GC)** cycles. This "stop-the-world" effect, combined with the 10 ms simulated GPU processing delay, leads to significant memory fragmentation. Consequently, the system experiences severe **tail latency escalation**, with maximum latency reaching **97.44 ms** in the baseline.
3. **Protocol and Scheduler Overhead:** Beyond application-layer data, each Unary RPC incurs HTTP/2 frame header overhead and a complete request-response round-trip. At high frequencies, the cumulative cost of managing thousands of concurrent handlers places immense strain on the Go runtime scheduler, further reducing throughput to only **564.7 requests/s**.

4.5 BASELINE WORKFLOW LOGIC

Figure 1 illustrates the synchronous flow. The linear dependency between the sensing device and the compute backend demonstrates why this model fails to scale to the demands of real-time HSR monitoring.

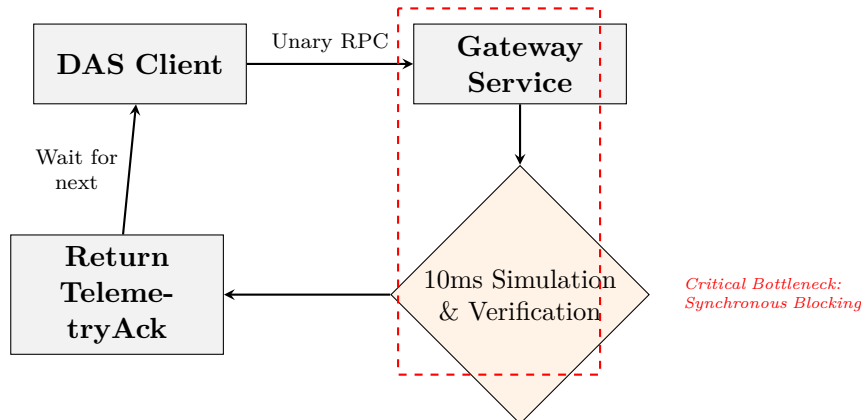


Figure 1: The synchronous execution path of the Baseline Gateway.

5 SYSTEM ARCHITECTURE: OPTIMIZED GATEWAY

5.1 MULTI-MODAL COMMUNICATION INTERFACE

The optimized gateway evolves from a rigid unary model to a flexible communication interface defined in `gateway_optimized.proto`. To support the high-throughput demands of HSR, the service implements a hybrid RPC architecture:

- **Client-Streaming (`StreamTelemetry`):** Enables the DAS client to push a continuous high-frequency data stream over a single long-lived connection, substantially reducing the per-packet HTTP/2 header overhead.
- **Server-Streaming (`SubscribeModeChanges`):** A dedicated asynchronous control channel. The server uses this stream to push state transitions to the client, effectively separating the **Control Plane** from the **Data Plane**.
- **Batch Processing (`PublishBatchTelemetry`):** Designed for HEARTBEAT mode, this unary RPC allows the client to upload multiple telemetry units in a single payload, minimizing server-side interrupts during idle periods.
- **Standard Unary (`PublishTelemetry`):** Retained for backward compatibility and single-packet diagnostic uploads.

5.2 ASYNCHRONOUS PROCESSING AND PRIORITY QUEUING

The core of the optimized server (`server.go`) is an **asynchronous producer-consumer architecture**. Unlike the baseline's blocking model, the gRPC handler here functions as a non-blocking producer.

1. **Ingestion-Time Evaluation:** Upon receiving a packet and before enqueueing, the gRPC handler immediately assesses the "loudness" (vibration amplitude). If a threshold is exceeded, the **Loudness-Driven State Machine** triggers a mode change, pushed via `SubscribeModeChanges`.
2. **Priority-Based Dispatching:** Incoming telemetry is directed to either a *High-Priority* channel (for real-time train signatures) or a *Low-Priority* channel (for background monitoring).
3. **Dedicated Worker Pools:** A fixed pool of worker goroutines (e.g., 4 high-priority, 8 low-priority) processes data from these channels. Workers perform the 10 ms simulated GPU processing independently of the ingestion layer, ensuring the front-end remains responsive.

5.3 MEMORY OPTIMIZATION: OBJECT POOLING

At 8 Gbps throughput, the frequency of memory allocations for telemetry objects and metric maps can trigger aggressive Garbage Collection (GC) cycles, leading to unpredictable tail latencies. The optimized gateway addresses this through the `pool.go` module:

Object pooling is implemented using `sync.Pool` for structures such as `Telemetry`, `Metric`, and `BatchTelemetry`. By reusing existing buffers instead of re-allocating them for each RPC, the gateway substantially reduces GC pressure. This optimization is crucial for maintaining stable performance during sustained high-throughput ingestion.

5.4 OPTIMIZED WORKFLOW LOGIC

Figure 2 visualizes the decoupled nature of the optimized gateway. Note that the loudness feedback originates at the ingestion stage, ensuring near-instantaneous mode transitions.

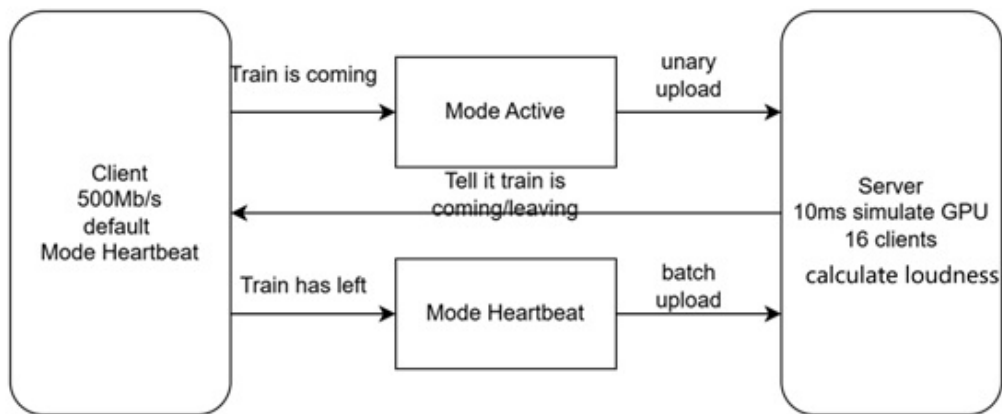


Figure 2: Optimized Workflow Logic.

6

BENCHMARKS AND EVALUATION

6.1 OBJECTIVES AND SCOPE

This evaluation seeks to stress-test the architectural limits of both the Baseline and Optimized gateways under extreme load conditions. By simulating 16 DAS devices concurrently

pushing data at 500 Mbps each—resulting in an aggregate ingestion demand of 8 Gbps—we quantify system behavior under saturation. Following the principles of tail latency management described by Dean and Barroso[4], our primary metrics include P95 and maximum latency alongside effective sample throughput. This focus on tail metrics is essential to identify performance variability and potential bottlenecks, such as Go’s GC pauses or resource contention, that are often masked by average values in high-frequency telemetry pipelines.

6.2 EXPERIMENTAL CONFIGURATION

- **Environment:** Benchmarks were conducted on an Intel Core i7-10700 machine with 32 GB of memory. To prevent network-stack interference from masking architectural bottlenecks, the load generator and gateways communicated via a high-performance local loopback interface.
- **Workload:** Each of the 16 DAS devices generated telemetry units consisting of 4 KB vibration slices. Each test run lasted 30 seconds, using a synchronized random seed to ensure consistent data patterns across all modes.
- **Simulation Parameters:** A fixed 10 ms “processing cost” was applied to every processed sample. In the Optimized gateway, the worker pool was limited to 4 high-priority and 8 low-priority goroutines to simulate constrained hardware-acceleration resources (e.g., shared GPUs or NPUs).

6.3 RESULTS AND COMPARATIVE ANALYSIS

Table 1 summarizes the performance across five operational modes under the 8 Gbps aggregate workload.

Table 1: Performance metrics under 8 Gbps aggregate workload (16 devices, 30 s). For Optimized-Batch, Optimized-Stream, and Optimized-Mixed, each message encapsulates 5 telemetry samples, so the effective per-sample throughput is $5\times$ the reported value.

Ingestion Mode	Messages	Avg Lat. (ms)	P95 Lat. (ms)	Max Lat. (ms)	Throughput (msg/s)
Baseline-Unary	14,312	30.23	42.67	124.37	477.1
Optimized-Unary	33,670	13.66	16.87	60.28	1,122.3
Optimized-Batch	11,936	33.82	46.74	89.25	397.9
Optimized-Stream	12,357	38.80	54.02	144.31	411.9
Optimized-Mixed	11,651	35.40	48.07	106.13	388.4

6.4 PERFORMANCE DISCUSSION

6.4.1 • SUPERIORITY OF ASYNCHRONOUS UNARY PROCESSING

The most notable result is the performance of **Optimized-Unary**, which achieves a throughput of **1,122.3 messages/s**—more than double that of the Baseline. In the **Baseline-Unary** mode, the average latency reaches 30.23 ms, despite each sample only nominally incurring a 10 ms processing cost. This discrepancy is largely due to *scheduling-induced queuing delay*: the synchronous handlers block for the full 10 ms (plus runtime scheduling overhead) before releasing their goroutines, causing a backlog of `PublishTelemetry` calls in the TCP and gRPC queues once the system approaches CPU saturation.

In contrast, **Optimized-Unary** registers an average latency of 13.66 ms, which is very close to the 10 ms processing baseline. This small gap (only ≈ 3.6 ms above the theoretical minimum) indicates that the *enqueue-and-return* strategy significantly reduces scheduling overhead under high load. Compared to the Baseline’s extra ≈ 20 ms of delay, the optimized asynchronous architecture demonstrates a much stronger capacity to suppress latency inflation and maximum latency when the CPU is heavily utilized.

6.4.2 • BATCHING AND STREAMING UNDER EXTREME LOAD

At first glance, the reported throughput (in messages/s) for **Optimized-Batch** and **Optimized-Stream** appears lower than that of **Optimized-Unary**. However, each batch/stream message encapsulates 5 individual telemetry samples. After accounting for this $5\times$ expansion, the *effective per-sample throughput* of the batching and streaming modes is comparable to, or higher than, the unary configuration, even though they operate with fewer messages per second at the RPC layer.

The remaining latency gap can be attributed to:

1. **Aggregation Latency:** **Optimized-Batch** introduces a collection window and additional buffer management. Under extreme concurrency, batch aggregation and queueing inevitably add delay on top of the per-batch processing time, increasing both average and P95 latencies.
2. **Stream Context Pressure:** While **StreamTelemetry** eliminates per-request header overhead, maintaining 16 long-lived, high-throughput streams under saturated CPU conditions increases scheduling pressure in the gRPC runtime. This results in a higher P95 latency (54.02 ms) and the largest maximum latency (144.31 ms) across all modes.

6.4.3 • RAILWAY PRIORITY AND MAXIMUM LATENCY CONTROL

The **Max Latency** column directly reflects the system’s ability to manage maximum latency under railway-style priority scenarios. In the Baseline mode, maximum latency spikes to 124.37 ms due to scheduling-induced queuing delay: once synchronous handlers are saturated, new requests are forced to wait behind the full 10 ms processing path plus accumulated queueing

delays. In contrast, Optimized-Unary limits maximum latency to 60.28 ms. Although higher than the 10 ms processing baseline, this peak under an 8 Gbps load demonstrates a substantial reduction in maximum latency compared to the uncontrolled tail latencies typically observed in saturated synchronous systems, effectively reducing the worst-case behavior by several times.

6.5 FUTURE WORK

While the current implementation delivers substantial performance improvements under an 8 Gbps aggregate workload, several avenues remain for further optimization and functional expansion:

1. **Hardware-Accelerated, Zero-Copy Ingestion:** To surpass the current throughput limit, it will likely be necessary to reimagine the interaction with the kernel networking stack. Future work includes exploring kernel-bypass and zero-copy technologies such as **eBPF/XDP** and **AF_XDP**, as well as NIC offload capabilities. Mapping network buffers directly into the gateway's ingestion paths would reduce CPU utilization and minimize scheduling-induced queuing delay, further narrowing the gap to the theoretical latency minimum.
2. **Adaptive Resource Orchestration:** In this study, the sizes of high-priority and low-priority worker pools are statically configured. A natural extension is to introduce an **adaptive autoscaling controller** that dynamically adjusts worker-pool and queue parameters based on observed load, latency SLOs, and accelerator utilization (e.g., GPU temperature and occupancy). Such a controller could enforce explicit latency/throughput targets while preventing priority inversion during rapidly changing train traffic patterns.
3. **Multi-Gateway and Edge-Cluster Coordination:** Real-world HSR deployments typically involve multiple gateways distributed along the track. Future work will extend the current single-node design to a **clustered edge fabric**, where gateways can share load, replicate critical events, and execute cooperative failover. This would enable geo-distributed DAS monitoring with end-to-end guarantees for both detection latency and system availability.
4. **Integration with Real Inference Pipelines:** The present evaluation models a fixed 10 ms processing cost. Replacing this abstraction with actual GPU- or NPU-accelerated inference pipelines (e.g., NVIDIA TensorRT, ONNX Runtime, or OpenVINO) will enable a more detailed study of how variable model latency, batching strategies, and accelerator scheduling policies interact with the gateway's back-pressure mechanisms and priority queues.
5. **Higher-Level Analytics and Feedback Loops:** Finally, the gateway can be extended from a pure transport and prioritization layer to a **closed-loop analytics node**. By incorporating simple on-device anomaly detectors or trend-analysis modules, the system

could refine its **ACTIVE/HEARTBEAT** policy based on longer-term track health indicators, enabling predictive maintenance workflows on top of low-level DAS telemetry.

REFERENCES

- [1] Invisensing, Distributed Acoustic Sensing (DAS) for Infrastructure Monitoring, CNRS Workshop Presentation, Sept. 2023. [Online]. Available: <https://www.reseau-capteurs.cnrs.fr/wp-content/uploads/2023/10/invisensing-CNRS-20230929.pdf>
- [2] Introduction to gRPC. Available: <https://grpc.io/docs/what-is-grpc/introduction/>
- [3] Ritu, S. Arora, A. Bhardwaj, A. Kukkar and S. Kaur, "A Comparative Analysis of Communication Efficiency: REST vs. gRPC in Microservice- Based Ecosystems," 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP), Sonipat, India, 2024, pp. 621-626, doi: 10.1109/INNOCOMP63224.2024.00107.
- [4] The Tail at Scale. Available: <https://www.barroso.org/publications/TheTailAtScale.pdf>